

Verification of Functional Programs

Defining Functions in Isabelle/HOL

Andrés Sicard-Ramírez

Universidad EAFIT

Semester 2026-1

Introduction

Introduction

In Isabelle/HOL the functions must be **total**.

Introduction

In Isabelle/HOL the functions must be **total**.

Functions can be defined using the following commands:

- **primrec**,
- **fun**,
- **function**.

Lexicographic Ordering

Definition

Let $(A, \preceq_A)$ and $(B, \preceq_B)$ be two posets. The **lexicographic ordering** $\preceq$ on $A \times B$ is defined by:

$$(a_1, b_1) \preceq (a_2, b_2) := a_1 \prec_A a_2 \text{ or } (a_1 = a_2 \text{ and } b_1 \preceq_B b_2).$$

The **primrec** Command

The **primrec** Command

Description

*“The keyword **primrec** indicates that the recursion is of a **particularly primitive kind** where each recursive call peels off a datatype constructor from one of the arguments. Thus the recursion always terminates, i.e. the function is total.”*
(Nipkow, Paulson, and Wenzel 2002, p. 11)

The **primrec** Command

Example (structural recursion on one argument)

The reverse function on polymorphic lists (Nipkow, Paulson, and Wenzel [2002] [2026](#)).

```
1 primrec rev :: "'a list  $\Rightarrow$  'a list" where  
2   "rev []          = []"  
3 | "rev (x # xs) = (rev xs) @ (x # [])"
```


The **primrec** Command

Example (structural recursion on one argument)

The reverse function on polymorphic lists (Nipkow, Paulson, and Wenzel [2002] 2026).

```
1 primrec rev :: "'a list  $\Rightarrow$  'a list" where  
2   "rev []          = []"  
3   | "rev (x # xs) = (rev xs) @ (x # [])"
```

The function is structurally recursive because in the (only) recursive call in line 3,

$xs \prec x \# xs$.

The **primrec** Command

Example (structural recursion on one of two arguments)

The append function on polymorphic lists (Nipkow, Paulson, and Wenzel [2002] [2026](#)).

```
1 primrec app :: "'a list  $\Rightarrow$  'a list  $\Rightarrow$  'a list" where  
2   "app []      ys = ys"  
3 | "app (x # xs) ys = x # (app xs ys)"
```

The **primrec** Command

Example (structural recursion on one of two arguments)

The append function on polymorphic lists (Nipkow, Paulson, and Wenzel [2002] 2026).

```
1 primrec app :: "'a list  $\Rightarrow$  'a list  $\Rightarrow$  'a list" where  
2   "app []      ys = ys"  
3   | "app (x # xs) ys = x # (app xs ys)"
```

The function is structurally recursive because in the (only) recursive call in line 3,

$$(xs, ys) \prec (x \# xs, ys).$$

The **primrec** Command

Counterexample (structural recursion on non-primitive patterns)

The Fibonacci function cannot be defined using **primrec**.

```
1 primrec fib :: "nat  $\Rightarrow$  nat" where  
2   "fib 0                = 1"  
3 | "fib (Suc 0)          = 1"  
4 | "fib (Suc (Suc n)) = fib n + fib (Suc n)"
```

Error:

Nonprimitive pattern in left-hand side at
fib (Suc 0) = 1

The **primrec** Command

Counterexample (structural recursion on multiple variable arguments)

The Ackermann–Péter function cannot be defined using **primrec**.

```
1 primrec ack :: "nat  $\Rightarrow$  nat  $\Rightarrow$  nat" where  
2   "ack 0      n      = Suc n"  
3 | "ack (Suc m) 0      = ack m 1"  
4 | "ack (Suc m) (Suc n) = ack m (ack (Suc m) n)"
```

Error:

More than one non-variable argument in left-hand side at

```
ack (Suc m) 0 = ack m 1
```

The **primrec** Command

Example (structural recursion on higher-order data types)

The add function on the so-called Brouwer ordinals.

```
1 datatype on = Z | S on | L "(nat  $\Rightarrow$  on)"
2
3 primrec add :: "on  $\Rightarrow$  on  $\Rightarrow$  on" where
4   "add a Z      = a"
5   | "add a (S b) = S (add a b)"
6   | "add a (L f) = L ( $\lambda$  n. add a (f n))"
```

(continued on next slide)

The **primrec** Command

Example (continuation)

The function is structurally recursive because (Abel and Altenkirch 2002)

(i) in the recursive call in line 5,

$$(a, b) \prec (a, S \ b),$$

(ii) in the recursive call in line 6,

$$(a, f \ n) \preceq (a, f) \prec (a, L \ f).$$

The **primrec** Command

Example (mutual recursion on mutual data types)

Arithmetic and booleans expressions defined by a mutual datatype (Nipkow, Paulson, and Wenzel [2002] 2026).

```
1 datatype 'a aexp = IF "'a bexp" "'a aexp" "'a aexp"
2                   | Sum "'a aexp" "'a aexp"
3                   | Diff "'a aexp" "'a aexp"
4                   | Var 'a
5                   | Num nat
6 and
7       'a bexp = Less "'a aexp" "'a aexp"
8           | And "'a bexp" "'a bexp"
9           | Neg "'a bexp"
```

(continued on next slide)

The **primrec** Command

Example (continuation)

Evaluation of arithmetic and booleans expressions by mutual recursion.

```
1  primrec
2    evala :: "'a aexp  $\Rightarrow$  ('a  $\Rightarrow$  nat)  $\Rightarrow$  nat" and
3    evalb :: "'a bexp  $\Rightarrow$  ('a  $\Rightarrow$  nat)  $\Rightarrow$  bool"
4  where
5    "evala (IF b a1 a2) env =
6      (if evalb b env then evala a1 env else evala a2 env)"
7  | "evala (Sum a1 a2)  env = evala a1 env + evala a2 env"
8  | "evala (Diff a1 a2) env = evala a1 env - evala a2 env"
9  | "evala (Var v)      env = env v"
10 | "evala (Num n)       env = n"
11
12 | "evalb (Less a1 a2) env = (evala a1 env < evala a2 env)"
13 | "evalb (And b1 b2)  env = (evalb b1 env  $\wedge$  evalb b2 env)"
14 | "evalb (Neg b)      env = ( $\neg$  evalb b env)"
```

The **fun** Command

The **fun** Command

Description

The **fun** command is a short form of the **function** command.[†]

$$\left[\begin{array}{l} \mathbf{fun} \ f \ :: \ \tau \\ \mathbf{where} \\ \quad \textit{equations} \\ \quad \vdots \end{array} \right] \equiv \left[\begin{array}{l} \mathbf{function} \ (\mathbf{sequential}) \ f \ :: \ \tau \\ \mathbf{where} \\ \quad \textit{equations} \\ \quad \vdots \\ \mathbf{by} \ \textit{pat_completeness} \ \mathbf{auto} \\ \mathbf{termination} \ \mathbf{by} \ \textit{lexicographic_order} \end{array} \right]$$

[†]Figure from (Krauss 2026, p. 4).

The **fun** Command

Example (structural recursion on non-primitive patterns)

The Fibonacci function (Nipkow, Paulson, and Wenzel [2002] 2026).

```
1 fun fib :: "nat  $\Rightarrow$  nat" where  
2   "fib 0                = 1"  
3 | "fib (Suc 0)          = 1"  
4 | "fib (Suc (Suc n)) = fib n + fib (Suc n)"
```

Found termination order: "size <*mlex*> {}"

The **fun** Command

Example (continuation)

The function is structurally recursive because

(i) in the second recursive call in line 3,

$$\text{Suc } n \prec \text{Suc } (\text{Suc } n),$$

(ii) in the first recursive call in line 3,

$$x \prec \text{Suc } n \prec \text{Suc } (\text{Suc } n).$$

The **fun** Command

Example (structural recursion on multiple variable arguments)

The Ackermann–Péter function.

```
1 fun ack :: "nat  $\Rightarrow$  nat  $\Rightarrow$  nat" where  
2   "ack 0      n      = Suc n"  
3   | "ack (Suc m) 0      = ack m 1"  
4   | "ack (Suc m) (Suc n) = ack m (ack (Suc m) n)"
```

Found termination order:

```
"( $\lambda$ p. size (fst p)) <*mlex*> ( $\lambda$ p. size (snd p)) <*mlex*> {}"
```

(continued on next slide)

The **fun** Command

Example (continuation)

The function is structurally recursive because

(i) in the recursive call in line 3,

$$(m, 1) \prec (\text{Suc } m, 0),$$

(ii) in the first recursive call in line 4,

$$(m, \text{ack } (\text{Suc } m) \ n) \prec (\text{Suc } m, \text{Suc } n),$$

(iii) in the second recursive call in line 4,

$$(\text{Suc } m, n) \prec (\text{Suc } m, \text{Suc } n).$$

The **fun** Command

Example (mutual recursion)

Functions even and odd by mutual recursion.

```
1 fun
2   even :: "nat  $\Rightarrow$  bool" and
3   odd  :: "nat  $\Rightarrow$  bool"
4 where
5   "even 0      = True"
6 | "odd  0      = False"
7 | "even (Suc n) = odd  n"
8 | "odd  (Suc n) = even n"
```

Found termination order: "case_sum size size <*mlex*> {}"

The **fun** Command

Example (higher-order recursion)

The mirror function on Rose trees.

```
1 datatype 'a rose_tree = Node "'a rose_tree list"
2
3 fun mirror :: "'a rose_tree  $\Rightarrow$  'a rose_tree" where
4   "mirror (Node ts) = Node (rev (map mirror ts))"
```

Found termination order: "size <*mlex*> {}"

The **function** Command

The **function** Command

Example (using a measure function)

The Fibonacci function.

```
1 function fib :: "nat  $\Rightarrow$  nat" where  
2   "fib 0                = 1"  
3 | "fib (Suc 0)          = 1"  
4 | "fib (Suc (Suc n)) = fib n + fib (Suc n)"  
5 by pat_completeness auto  
6 termination fib  
7   apply (relation "measure ( $\lambda$ n. n)")  
8   by auto
```

The **function** Command

Example (using a measure function)

The Ackermann–Péter.

```
1 function ack :: "nat  $\Rightarrow$  nat  $\Rightarrow$  nat" where
2   "ack 0      n      = Suc n"
3 | "ack (Suc m) 0      = ack m 1"
4 | "ack (Suc m) (Suc n) = ack m (ack (Suc m) n)"
5   by pat_completeness auto
6 termination
7   apply (relation "measures [fst , snd]")
8   by auto
```

The **function** Command

Example (mutual recursion)

Functions even and odd by mutual recursion.

```
1 fun
2   even :: "nat  $\Rightarrow$  bool" and
3   odd  :: "nat  $\Rightarrow$  bool"
4 where
5   "even 0      = True"
6 | "odd  0      = False"
7 | "even (Suc n) = odd  n"
8 | "odd  (Suc n) = even n"
9 by pat_completeness auto
10 termination
11 by (relation "measure ( $\lambda x$ . case x of Inl n  $\Rightarrow$  n | Inr n  $\Rightarrow$  n)") auto
```

“Isabelle internally creates a single function operating on the sum type $\text{nat} + \text{nat}$... Consequently, termination has to be proved simultaneously for both functions, by specifying a measure on the sum type.” (Krauss 2026, p. 7)

The **function** Command

Example

An example where the lexicographic order fails for the termination proof (Krauss [2026](#)).

```
1 function sum :: "nat  $\Rightarrow$  nat  $\Rightarrow$  nat" where  
2   "sum i N = (if i > N then 0 else i + sum (Suc i) N)"  
3 by pat_completeness auto  
4 termination sum  
5   apply (relation "measure ( $\lambda(i,N). N + 1 - i$ )")  
6   by auto
```

The **function** Command

Example

An example where using something different to `pat_completeness` (Krauss 2026).

```
1 function gcd :: "nat  $\Rightarrow$  nat  $\Rightarrow$  nat" where
2   "gcd x 0 = x"
3 | "gcd 0 y = y"
4 | "x < y  $\implies$  gcd (Suc x) (Suc y) = gcd (Suc x) (y - x)"
5 | "~ x < y  $\implies$  gcd (Suc x) (Suc y) = gcd (x - y) (Suc y)"
6 by (atomize_elim, auto, arith)
7 termination by lexicographic_order
```

References

References

- Andreas Abel and Thorsten Altenkirch (2002). A Predicative Analysis of Structural Recursion. *Journal of Functional Programming* 12.1, pp. 1–41. DOI: [10.1017/S0956796801004191](https://doi.org/10.1017/S0956796801004191) (cit. on p. 15).
- Alexander Krauss (Jan. 18, 2026). Defining Recursive Functions in Isabelle/HOL. URL: <https://isabelle.in.tum.de/dist/Isabelle2025-2/doc/functions.pdf> (cit. on pp. 19, 29–31).
- Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel (2002). Isabelle/HOL. A Proof Assistant for Higher-Order Logic. Vol. 2283. *Lecture Notes in Computer Science*. Springer. DOI: [10.1007/3-540-45949-9](https://doi.org/10.1007/3-540-45949-9) (cit. on pp. 7, 33).
- [2002] (Jan. 18, 2026). Isabelle/HOL. A Proof Assistant for Higher-Order Logic. Updated version of (Nipkow, Paulson, and Wenzel 2002) matching changes in Isabelle/HOL. URL: <https://isabelle.in.tum.de/dist/Isabelle2025-2/doc/tutorial.pdf> (cit. on pp. 8–11, 16, 20).